

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»

Отчёт по рубежному контролю №2 по курсу
Парадигмы и конструкции языков программирования
ГУИМЦ

Исполнитель: **Кузнецов Н. В.**
студент группы
ИУ5Ц-54Б

Преподаватель: **Гапанюк Ю. Е.**

Москва, МГТУ – 2025

Условия рубежного контроля №2 по курсу ПиК ЯП

Рубежный контроль представляет собой разработку тестов на языке Python.

1) Проведите рефакторинг текста программы рубежного контроля №1 таким образом, чтобы он был пригоден для модульного тестирования.

2) Для текста программы рубежного контроля №1 создайте модульные тесты с применением TDD - фреймворка (3 теста).

Текст программы:

Файл rk1_refactored.py

```
# rk1_refactored.py
from operator import itemgetter

class Table:
    def __init__(self, id, name, row_count, db_id):
        self.id = id
        self.name = name
        self.row_count = row_count
        self.db_id = db_id

class Database:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class TableDatabase:
    def __init__(self, db_id, table_id):
        self.db_id = db_id
        self.table_id = table_id

def get_one_to_many(dbs, tables):
    return [(t.name, t.row_count, d.name)
            for d in dbs for t in tables if t.db_id == d.id]

def get_many_to_many(dbs, tables, tables_dbs):
    temp = [(d.name, td.db_id, td.table_id)
            for d in dbs for td in tables_dbs if d.id == td.db_id]
    return [(t.name, t.row_count, db_name)
            for db_name, db_id, table_id in temp
            for t in tables if t.id == table_id]

def task1(one_to_many):
    return list(filter(lambda i: i[0].endswith('об'), one_to_many))

def task2(dbs, one_to_many):
    res = []
    for d in dbs:
        d_tables = list(filter(lambda i: i[2] == d.name, one_to_many))
        if d_tables:
            avg = round(sum(r for _, r, _ in d_tables) / len(d_tables), 2)
            res.append((d.name, avg))
    return sorted(res, key=itemgetter(1))

def task3(dbs, many_to_many):
    res = {}
    for d in dbs:
        if d.name.startswith('А'):
```

```

        tables = [name for name, _, db in many_to_many if db == d.name]
        res[d.name] = tables
    return res

def main():
    dbs = [
        Database(1, 'Активная база'),
        Database(2, 'Архивная база данных'),
        Database(3, 'Тестовая'),
        Database(4, 'Аналитическая база данных'),
        Database(5, 'Основная база'),
    ]

    tables = [
        Table(1, 'Пользователей', 1000, 1),
        Table(2, 'Заказов', 5000, 2),
        Table(3, 'Товаров', 800, 2),
        Table(4, 'Аналитика продаж', 300, 4),
        Table(5, 'Логов', 1200, 5),
    ]

    tables_dbs = [
        TableDatabase(1, 1),
        TableDatabase(2, 2),
        TableDatabase(2, 3),
        TableDatabase(4, 4),
        TableDatabase(5, 5),
        TableDatabase(1, 2),
        TableDatabase(2, 1),
    ]

    one_to_many = get_one_to_many(dbs, tables)
    many_to_many = get_many_to_many(dbs, tables, tables_dbs)

    print('Задание Д1:', task1(one_to_many))
    print('Задание Д2:', task2(dbs, one_to_many))
    print('Задание Д3:', task3(dbs, many_to_many))

if __name__ == '__main__':
    main()

```

Файл test_rk1.py

```
# test_rk1.py
import unittest
from rk1_refactored import *

class TestRK1(unittest.TestCase):
    def setUp(self):
        self.dbs = [
            Database(1, 'Активная база'),
            Database(2, 'Архивная база данных'),
            Database(3, 'Тестовая'),
        ]

        self.tables = [
            Table(1, 'Пользователей', 1000, 1),
            Table(2, 'Заказов', 5000, 2),
            Table(3, 'Товары', 800, 2),
        ]

        self.tables_dbs = [
            TableDatabase(1, 1),
            TableDatabase(2, 2),
            TableDatabase(2, 3),
        ]

        self.one_to_many = get_one_to_many(self.dbs, self.tables)
        self.many_to_many = get_many_to_many(self.dbs, self.tables,
self.tables_dbs)

    def test_task1(self):
        result = task1(self.one_to_many)
        self.assertEqual(len(result), 1)
        self.assertTrue(all(i[0].endswith('ов') for i in result))

    def test_task2(self):
        result = task2(self.dbs, self.one_to_many)
        self.assertEqual(len(result), 2) # 2 БД имеют таблицы
        self.assertLessEqual(result[0][1], result[1][1]) # отсортированы

    def test_task3(self):
        result = task3(self.dbs, self.many_to_many)
        self.assertEqual(len(result), 2) # 2 БД начинаются на 'А'
        self.assertIn('Активная база', result)
        self.assertIn('Архивная база данных', result)

if __name__ == '__main__':
    unittest.main()
```

Результат выполнения программы:

===== test session starts =====

platform win32 -- Python 3.12.3, pytest-9.0.2, pluggy-1.6.0 --

C:\Users\nikit\AppData\Local\Programs\Python\Python312\python.exe

cachedir: .pytest_cache

rootdir: C:\Users\nikit\YandexDisk\Учеба 2026 Бауманка\ПКЯП\PCPL-labs-2025\rk2

collected 3 items

test_rk1.py::TestRK1::test_task1 PASSED [33%]

test_rk1.py::TestRK1::test_task2 PASSED [66%]

test_rk1.py::TestRK1::test_task3 PASSED [100%]

===== 3 passed in 0.03s =====